

A citation can be real, from a real page, and still wrong.

TraceLine is a maintenance assistant whose job is not to answer faster, but to **refuse to answer when it cannot do it honestly**. It checks whether the evidence it retrieved actually applies to the machine in front of you — and when it cannot tell, it says which single field would settle it.

8

synthetic records, BM25-indexed

5

applicability verdicts, metadata-derived

41

unit tests over the rule set

1

third-party runtime dependency

PROJECT
TraceLine

BUILD
Browser prototype · no backend

SUBTITLE
Configuration-aware maintenance evidence

CORPUS
Synthetic throughout

THE PROBLEM

The wrong-revision answer looks exactly like the right one.

A technician asks what a fault code means. Search returns a real document, with a real page number, stating a real answer — for a different firmware revision of that machine. **The citation is the problem.** It makes a wrong answer look verified, and someone who would have double-checked an unsourced answer does not double-check a cited one.

What a conventional assistant returns

Answered · 1 source

SRC-021 · p. 11 · table 37.63

On firmware 2.3.x, the table maps F12 to supply input below a configured threshold.

Troubleshooting table

firmware 2.3.8

Real document. Real page. Correct content — for firmware 2.3.8. Confirm 2.4.1 on this asset and a different record applies. Nothing in the sentence says so.

- 01

Applicability is invisible in the text

Whether a document covers this model, firmware and supply class lives in metadata and revision tables — not in the sentence that answers the question.
- 02

Documents disagree and never say so

Two references describe the same fault code correctly, for different releases. Neither one references the other.
- 03

The deciding field is often unknown

The technician may not know the installed firmware. A system that quietly assumes a value has invented its most important input.

Applicability is a blocking condition, not a metadata field to display.

Most systems retrieve, rank, and answer. TraceLine inserts one step: before a record can support a statement, its **declared scope** must cover the **confirmed** asset configuration. Fail that and the record stays on screen, labelled with why it was excluded. Cannot evaluate it, and no answer is produced at all.

- 01

Retrieve
BM25 over the corpus. A query naming a fault code restricts citable records to carriers of that code.
- 02

Evaluate scope
Compare each record's declared model, firmware and supply against confirmed context. Five possible verdicts.
- 03

Decide, or decline
Scope the answer, surface the disagreement, or abstain and name the missing field. Never guess.

FOUR THINGS THE VERDICT CAN BE, AND WHAT THE INTERFACE SAYS

VERDICT	BADGE SHOWN	WHAT IT MEANS
conflict	Clarification needed	Sources disagree and the deciding field is unconfirmed. Both cited, neither selected.
scoped	Context applied · scoped answer	One record's declared scope covers the confirmed context. Others excluded with reasons.
abstain	Insufficient evidence	No record's scope matches, or nothing addresses the question. Missing evidence named.
model-scoped-abstain	Out of model scope	Evidence exists, but for a different equipment platform.

Asked about F12 with firmware unconfirmed, it cites both and picks neither.

Evidence ledger • AX-480 • asset 00481		Clarification needed
SRC-014 • p. 4 • region 2		7.71
On firmware 2.4.x, F12 is recorded when the controller cannot confirm the configured supply class during startup diagnostics.		
Service note firmware 2.4.1		
SRC-021 • p. 11 • table 3		7.63
On firmware 2.3.x, the table maps F12 to supply input below a configured threshold.		
Troubleshooting table firmware 2.3.8		
Needed to resolve: confirm controller firmware from the asset or release record.		
Also retrieved • not cited		
SRE-071 • entry 14 • field note		7.12
A technician recorded a prior F12 event on this asset and noted that it cleared after a site power interruption.		
Shown and labelled, excluded from both the disagreement and cited support. A field note is not a controlled document.		

Confirm 2.4.1 and the scope resolves. Confirm 2.3.8 and it inverts.

Exclusion is **visible, with its reason** — never a silent drop. No record is ever both cited and excluded in the same answer.

Firmware 2.4.1 · confirmed		Context applied · scoped answer
SRC-014 · p. 4 · region 2		7.71
Service note · firmware 2.4 series		
Applies to confirmed context		
SRC-088 · p. 5 · callout B		6.00
Control wiring drawing · supply sensing		
Applies to confirmed context		
SRC-021 · p. 11 · table 3		7.63
Troubleshooting table · legacy release		
Different firmware scope		

Firmware 2.3.8 · confirmed		Context applied · scoped answer
SRC-021 · p. 11 · table 3		7.63
Troubleshooting table · legacy release		

A photo suggests. A technician decides.

Client-side OCR reads a nameplate image and writes firmware into the asset context as an `ocr`-sourced **suggestion**. The answer does not change. Only explicit confirmation promotes it to confirmed context — so machine perception informs the human and never substitutes for them.

Nameplate capture

OCR suggestion · unconfirmed

SUGGESTED

Firmware **2.4.1**, with the OCR confidence the engine reported

IF THE READ IS DIRTY

Nameplate OCR drops and invents separators, so a run like **241** is matched against known releases and labelled *reconstructed* — never resolved into a version that does not exist. A match against nothing known returns no suggestion at all.

LEDGER

Unchanged. Still **Clarification needed**, still asking for firmware.

An OCR reading is a suggestion. TraceLine will not scope an answer to a value the technician has not confirmed.

FIELD PROVENANCE IS TYPED, AND ONLY TWO TYPES COUNT AS CONFIRMED

SOURCE	ORIGIN	CAN SCOPE AN ANSWER
system	Asset record	Yes
user	Technician confirmed	Yes
ocr	Image read	No
unknown	Absent	No

Unconfirmed is a first-class state. “No verdict available” is represented distinctly from “fails the check”. Collapsing those two is what makes a system either over-cite or silently drop valid evidence.

Two different ways to have no answer, and both are stated.

The record exists — its scope does not match

Insufficient evidence

ASKED	What is fault F18?
CONTEXT	Firmware 2.4.1, confirmed
RETURNED	No claims. F18 appears only in the 2.3.8 table.
MISSING	A document whose stated firmware scope includes 2.4.1.

SRC-021 • p. 11 • table 3

retrieved

Retrieved, ranked, then excluded — still on screen with its reason.

Different firmware scope

Nothing addresses the question at all

Insufficient evidence

ASKED	What is the safe torque value for this motor?
RETURNED	No claims, no citations.
MISSING	An approved equipment document covering the requested topic; the exact asset configuration the question depends on.

Asking about a fault code absent from the corpus also abstains. It will not cite a record because the word “fault” appears in it.

Absence is a valid answer. An assistant that says *I need one more fact* is worth more to a technician than one that answers everything confidently and is occasionally, invisibly wrong.

DEMONSTRATED, NOT ASSERTED

Turn the applicability check off and the failure reproduces on demand.

The prototype ships its own control condition. One toggle removes the applicability filter, and the same F12 question — on an **AX-480** asset with firmware 2.4.1 confirmed — cites four records. The top-ranked one is a troubleshooting table for the **AX-320** platform.

SAME QUESTION, SAME CONFIRMED CONTEXT, FILTER OFF

RANK	RECORD	DECLARED MODEL	DECLARED FIRMWARE	BM25
1	SRC-052 Troubleshooting table · AX-320 platform	AX-320	1.9.4	7.77
2	SRC-014 Service note · firmware 2.4 series	AX-480	2.4.1	7.71
3	SRC-021 Troubleshooting table · legacy release	AX-480	2.3.8	7.63
4	SRC-088 Control wiring drawing · supply sensing	AX-480	2.4.1	6.00

Lexical similarity ranks the wrong-platform table first, because it is the document that talks about F12 most directly. That is not a retrieval bug to tune away — it is the expected behaviour of similarity search, and the reason applicability has to be a separate gate. Turn the check back on and SRC-052 is excluded, labelled **Different equipment model**.

HOW IT IS BUILT

A DOM-free rule engine, tested directly, with one runtime dependency.

$$\text{score}(t) = \text{idf}(t) \cdot (tf \cdot (k_1 + 1)) \frac{tf + k_1 \cdot (1 - b + b \cdot \text{len}/\text{avglen})}{k_1 + 1} \quad k_1 = 1.4 \quad b = 0.72$$
$$\text{idf}(t) = \ln(1 + (N - n + 0.5) / (n + 0.5))$$

BM25 written from scratch so the scoring is inspectable and the tests can assert on term frequencies, document lengths and the idf map directly. Scores are shown per record in the interface rather than hidden behind a relevance bar.

APPLICABILITY, EVALUATED IN THIS ORDER

#	CHECK	VERDICT WHEN IT FAILS
1	Record model ≠ confirmed model	out-of-model
2	Record supply ≠ confirmed supply	out-of-supply
3	Firmware not confirmed	unresolved
4	Confirmed firmware outside record scope	out-of-firmware
5	All checks pass	applicable

Order matters. **unresolved** is returned *before* the firmware comparison, so a missing value can never be read as a failed check.

STACK	Hand-written HTML and CSS, native ES modules, no framework. Tesseract.js 5.1.0 from a CDN is the only third-party runtime dependency.
NOT PRESENT	No <code>package.json</code> , lockfile, <code>node_modules</code> , bundler, transpiler, CSS preprocessor, backend, database, API key, or environment variable.
TESTS	<code>node --test</code> · 41 tests · ~126 ms · named for the failure each one forbids. The engine imports no DOM, so the rules are tested without a browser.

What is real in this build, and what is not.

A prototype that overstates itself is worth less than one that states its edges. These are the edges.

Working	In the build
BM25 retrieval with visible per-record scores · exact-identifier handling · five-verdict applicability · metadata-derived conflict detection · typed field provenance · client-side OCR with noisy-read reconstruction · documented claims separated from field observations · structured abstention that names what is missing · citation links that open the cited region · keyboard-operable evidence list, ARIA live regions, layout holding at 400 px.	
Not built	Stated, not implied
No semantic embeddings or vector store — retrieval is lexical only. No PDF parsing, no real table or diagram extraction, no LLM generation, no backend, no authentication, no persistence. The diagram regions are human-authored fixtures, not computer-vision output. No accuracy has been measured ; the idea-phase targets are hypotheses to test, not results.	

Every equipment identifier, document title, excerpt, date and revision is **synthetic**. Nothing is reproduced from manufacturer documentation, and no integration or endorsement is claimed. TraceLine does not control equipment, diagnose physical faults, or authorise work — the interface says so on every screen.

The rules are pure functions over record metadata.

Nothing in the approach is specific to drives or firmware. The same rules hold when the corpus becomes an ingestion pipeline instead of a fixture array, and they apply to any domain where evidence is scoped — **model, serial range, supply class, region, effective date** — and where citing out-of-scope evidence is worse than citing nothing.

01

Replace the fixture corpus with ingested records carrying the same declared-scope fields. The verdict logic does not change.

02

Every exclusion is derived from declared metadata, so it is auditable after the fact — the minimum bar for regulated maintenance work.

03

Next step: run the workflow against approved documents, with engineers reviewing the applicability labels.

LIVE PROTOTYPE	juliaj3.github.io/traceline-prototype
SOURCE	github.com/JuliaJ3/traceline-prototype
DOCUMENTATION	PROJECT_SUMMARY.md · TECHNICAL_DOCUMENTATION.md · EVALUATION.md